

Advanced Algorithms

Mock Exam - Structure-Matched Practice Version 1

Name: _____ Student ID: _____

Date: _____ Duration (suggested) minutes _____

Grading: Multiple Choice: correct +0.3, wrong -0.1. True/False: correct +0.2, wrong -0.1. Leave blank = 0.

Instructions: Answer each question unambiguously. For MCQ, select exactly one option (A-D).

Part I - Multiple Choice (20 questions)

1. Consider the terms *deterministic*, *non-deterministic*, and *randomized* algorithms. Which statement best matches the lecture definitions?

- A. A deterministic algorithm may return different outputs on the same input depending on random bits.
- B. A non-deterministic algorithm may behave differently on the same input due to auxiliary factors (e.g., random values), often to obtain approximate solutions.
- C. A randomized algorithm is deterministic, but its analysis uses probability.
- D. A non-deterministic algorithm always guarantees the correct answer.

2. Which pair correctly matches the two main classes of randomized algorithms discussed (with their key guarantee)?

- A. Monte Carlo: always correct, but running time is random.
- B. Las Vegas: may return an incorrect answer, but running time is bounded.
- C. Monte Carlo: running time is fixed/limited, but may be incorrect.
- D. Las Vegas: running time is fixed/limited, but may be incorrect.

3. A Monte Carlo algorithm fails (returns the wrong answer) with probability 0.25 on each *independent* run. If you repeat it 5 times and only accept the answer if all 5 runs agree, what is an upper bound on the probability that you still accept a wrong answer?

- A. 0.25^5
- B. 5×0.25
- C. $0.25/5$
- D. $1 - (1-0.25)^5$

4. Fermat's primality test can label a composite integer as "probably prime". What is the primary structural reason this can happen?

- A. Because modular exponentiation is too slow to compute exactly.
- B. Because some composite numbers (Carmichael numbers) satisfy Fermat's congruence for many (or all) bases that are coprime to them.
- C. Because Fermat's theorem only holds for even integers.
- D. Because choosing a random base always has probability 1/2 of being misleading.

5. In the "randomized search" example where an array of n elements contains half 'a' and half 'b', we repeatedly sample a random index until we find an 'a'. What is the expected number of samples (ignoring the cost of random-number generation)?

- A. 1
- B. 2
- C. $n/2$

D. n

6. Randomized Quicksort is typically obtained from standard Quicksort by:
- A. Choosing the pivot uniformly at random (or randomly permuting the array) before partitioning.
 - B. Partitioning into three parts using median-of-medians.
 - C. Always choosing the smallest element as pivot.
 - D. Stopping recursion early when subarrays are unbalanced.
7. In the 0-1 Knapsack problem, a simple greedy heuristic sorts items by value/weight ratio and takes items that fit. Why can this heuristic be arbitrarily bad?
- A. Because sorting by ratio is $O(n^2)$ in the worst case.
 - B. Because 0-1 Knapsack has discrete choices; a locally best ratio item can block a much better combination later.
 - C. Because ratios cannot be computed for integer weights.
 - D. Because the heuristic always chooses the heaviest item first.
8. For the *fractional* (continuous) Knapsack problem, sorting items by value/weight ratio and taking as much as possible from the best ratios is:
- A. Always optimal.
 - B. Always suboptimal compared to dynamic programming.
 - C. Optimal only when all weights are equal.
 - D. Optimal only when all values are equal.
9. In the fractional Knapsack example with $W=50$ and items $(w,v) = (10,60), (20,100), (30,120)$, what total value does the greedy ratio strategy obtain?
- A. 160
 - B. 220
 - C. 240
 - D. 280
10. For Activity Selection (interval scheduling), the greedy rule that is proven optimal selects next the activity with:
- A. Earliest start time.
 - B. Shortest duration.
 - C. Earliest finish time among compatible activities.
 - D. Largest number of conflicts.
11. Kruskal's algorithm for MST repeatedly:
- A. Adds the cheapest edge in the whole graph, even if it creates a cycle, provided it is light.
 - B. Adds the cheapest edge that does **not** create a cycle.
 - C. Adds the heaviest edge that reduces the number of components.
 - D. Grows a single tree by always adding the closest vertex to the source.
12. Dijkstra's algorithm for single-source shortest paths requires which condition on edge weights for correctness?
- A. All weights are distinct.
 - B. All weights are non-negative.
 - C. All weights are integers.
 - D. The graph must be complete.
13. A Morris-style probabilistic counter stores an integer X and uses an estimator of the form $2^X - 1$. If the counter currently stores $X=10$, the estimate is:
- A. 10
 - B. 1000

- C. 1023
- D. 2047

14. A Bloom filter with m bits, n inserted items, and k hash functions has approximate false-positive probability:

- A. $(1 - e^{-kn/m})^k$
- B. $e^{-kn/m}$
- C. $(1 - kn/m)^k$
- D. $1 - (1 - 1/m)^{kn}$

15. A Bloom filter has $m=1000$ bits and stores $n=100$ items. The near-optimal choice of k is approximately:

- A. 1
- B. 3
- C. 7
- D. 100

16. Using $m=1000$, $n=100$, $k=7$, the approximate Bloom filter false-positive rate (using $(1 - e^{-kn/m})^k$) is closest to:

- A. 0.8%
- B. 8%
- C. 18%
- D. 80%

17. Which statement is true about Cuckoo filters compared to standard Bloom filters?

- A. Cuckoo filters cannot support deletions without rebuilding the structure.
- B. Cuckoo filters can support deletions (because they store fingerprints in buckets), while Bloom filters typically cannot delete without counting.
- C. Cuckoo filters have false negatives by design, even when inserts succeed.
- D. Bloom filters always use less memory for the same false-positive rate.

18. In a Count-Min Sketch with error parameters (ϵ, δ) , a standard parameter choice is $w = \lceil e/\epsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$. For $\epsilon=0.01$ and $\delta=0.05$, (w,d) is:

- A. (100, 2)
- B. (272, 3)
- C. (272, 20)
- D. (300, 5)

19. Which property is characteristic of Count-Min Sketch estimates for frequencies?

- A. They are unbiased and can be below the true count.
- B. They never exceed the true count.
- C. They are upper bounds (can overestimate) due to hash collisions.
- D. They are exact if $d \geq 2$.

20. Misra-Gries with $(k-1)$ counters guarantees that any item with frequency greater than:

- A. n/k
- B. k/n
- C. $n/(k-1)$
- D. $n/2$

Part II - True / False (20 statements)

Indicate unequivocally whether each statement is True (T) or False (F).

- 1.** A deterministic algorithm can return different answers on the same input because its behaviour depends on random bits.
- 2.** A Las Vegas algorithm is always correct when it returns an answer, but its running time is a random variable.
- 3.** Repeating an independent Monte Carlo algorithm and taking a majority vote reduces error probability exponentially in the number of repetitions.
- 4.** Fermat's primality test never makes mistakes for composite numbers that are not Carmichael numbers.
- 5.** In randomized Quicksort, the worst-case running time is still $\Theta(n^2)$; randomization changes the expected running time.
- 6.** For 0-1 Knapsack, a greedy strategy that is optimal for fractional Knapsack is guaranteed to be within a constant factor of optimal for every instance.
- 7.** Dynamic programming relies on optimal substructure and overlapping subproblems; memoization is one way to exploit this.
- 8.** In dynamic programming, "pruning" plays the same role as in backtracking: it safely discards branches using bounds.
- 9.** Kruskal's algorithm can add an edge even if it creates a cycle, as long as the edge has minimum weight among remaining edges.
- 10.** If a graph contains negative-weight edges, Dijkstra's algorithm can return incorrect shortest paths.
- 11.** A Bloom filter can produce false positives but (assuming correct inserts) it does not produce false negatives.
- 12.** In a Bloom filter, increasing the number of inserted elements n (with m and k fixed) typically increases the false-positive probability.
- 13.** The near-optimal number of hash functions for a Bloom filter is $k \approx (m/n) \ln 2$.
- 14.** A counting Bloom filter replaces each bit with a small counter, enabling deletions at the cost of more space.
- 15.** A Cuckoo filter stores full keys in its buckets, so its false-positive rate is zero.
- 16.** Count-Min Sketch can underestimate the true frequency of an item due to collisions.
- 17.** Count-Min Sketch uses multiple hash tables so that taking the minimum across rows reduces overestimation bias from collisions.
- 18.** Misra-Gries can be used to find heavy hitters in a stream using $O(k)$ memory, for a chosen parameter k .
- 19.** Any item that appears more than n/k times in the stream is guaranteed to be reported by Misra-Gries (with $k-1$ counters).
- 20.** Morris probabilistic counting stores an exact count in binary; its randomness is only used to speed up increments.

Solutions (for self-check)

Answer Key - Part I (MCQ)

1:B 2:C 3:A 4:B 5:B 6:A 7:B 8:A 9:C 10:C 11:B 12:B 13:C 14:A 15:C 16:A 17:B 18:B 19:C 20:A

Answer Key - Part II (True/False)

1:F 2:T 3:T 4:F 5:T 6:F 7:T 8:F 9:F 10:T 11:T 12:T 13:T 14:T 15:F 16:F 17:T 18:T 19:T 20:F

Brief Explanations

Part I - Multiple Choice

1 (Answer B). Non-determinism here refers to variability caused by auxiliary factors (e.g., random choices) rather than producing a single, fixed output for a fixed input. (A,C) describe randomization or probabilistic analysis, not the definition; (D) is false because non-determinism does not imply correctness.

2 (Answer C). Monte Carlo has bounded/known running time but may be wrong with small probability; Las Vegas is always correct but has random running time. (A,B,D) swap these properties.

3 (Answer A). If each run is wrong with probability 0.25 and runs are independent, all 5 being wrong has probability $(0.25)^5$. The other options correspond to linear scaling or the probability of at least one failure, not "still wrong after all repeats".

4 (Answer B). Carmichael numbers are composite but satisfy $a^{n-1} \equiv 1 \pmod{n}$ for many/all coprime bases, so Fermat can be fooled. (A) is about efficiency, not correctness; (C) is nonsense; (D) asserts a fixed probability that isn't generally true.

5 (Answer B). Each sample hits an 'a' with probability $1/2$, so the number of trials is geometric with mean $1/(1/2)=2$. (A) would require probability 1; (C,D) confuse expectation with worst-case scaling.

6 (Answer A). Randomization is typically done by choosing a random pivot (or shuffling first), which yields expected $\Theta(n \log n)$. (B) is a deterministic pivot-selection method; (C) is worst-case; (D) changes the algorithm and breaks correctness/analysis.

7 (Answer B). Discrete (0-1) choices create "packing" effects: a high ratio item can prevent selecting a combination with higher total value. (A) is irrelevant; (C) ratios are fine; (D) misstates the heuristic.

8 (Answer A). With fractional amounts allowed, the exchange argument shows taking best ratios first is optimal. The other options add restrictions that aren't needed or claim DP is required.

9 (Answer C). Greedy takes (10,60) and (20,100) fully (weight 30), then $20/30$ of the last item (value $120 \times 2/3 = 80$), totaling 240. Other values correspond to taking fewer items or an impossible value above the fractional optimum.

10 (Answer C). Choosing earliest finishing compatible activity leaves maximal room for the remaining schedule. (A,B) fail on classic counterexamples; (D) is the opposite of what we want.

11 (Answer B). Kruskal adds the lightest edge that does not form a cycle (using Union-Find). (A) is the common trap; (C) is unrelated; (D) describes Prim/Dijkstra-like growth.

12 (Answer B). Non-negative weights are needed because Dijkstra assumes once a node is extracted as minimum-distance, that distance is final. Negative edges can later create shorter paths, invalidating that assumption.

13 (Answer C). Estimator is $2^X - 1$, so $2^{10} - 1 = 1023$. (D) would be $2^{11}-1$; (A,B) are unrelated.

14 (Answer A). Standard approximation: after kn bit-sets, fraction of 0s is $\approx e^{-kn/m}$, so false positive $\approx (1 - e^{-kn/m})^k$. (B) is probability a bit remains 0; (C) is a rough linearization; (D) is close to the exact "bit remains 0" term but not the final FP.

15 (Answer C). $k_{\text{opt}} \approx (m/n) \ln 2 = (1000/100) \times 0.693 \approx 6.93 \approx 7$. (A,B) are too small; (D) is far too large and would saturate the filter.

16 (Answer A). Compute $kn/m = 700/1000 = 0.7$, $e^{-0.7} \approx 0.4966$, so $1 - e^{-0.7} \approx 0.5034$, $(0.5034)^7 \approx 0.0082 \approx 0.8\%$. The other options are off by orders of magnitude.

17 (Answer B). Cuckoo filters store short fingerprints in buckets and can delete by removing the fingerprint; Bloom filters need counting to delete. (A) is wrong; (C) false negatives aren't intended when inserts succeed; (D) is not universally true (depends on target FP and design).

18 (Answer B). $w = \lceil e/\epsilon \rceil = \lceil 2.718/0.01 \rceil = 272$ and $d = \lceil \ln(1/\delta) \rceil = \lceil \ln 20 \rceil = 3$. The other pairs misuse $\ln$ vs $1/\delta$ or pick arbitrary numbers.

- 19 (Answer C).** Due to collisions, each row count is $\geq$ true count; taking the minimum keeps an upper bound while reducing collision noise. (A,B,D) contradict this core property.
- 20 (Answer A).** With $k-1$ counters, any item with frequency $> n/k$ must survive the elimination process and be present among the candidates. The other thresholds are incorrect or too weak/strong.

Part II - True/False

- 1 (Answer F).** False: deterministic means the same input always yields the same behaviour/output; random bits would make it randomized, not deterministic.
- 2 (Answer T).** True: Las Vegas algorithms may take variable time, but when they return, the answer is correct (they may rerun/restart internally).
- 3 (Answer T).** True: independence plus repetition lets the failure probability drop like p^r (or via majority vote), i.e., exponentially in repetitions.
- 4 (Answer F).** False: even for non-Carmichael composites, Fermat can still err if you pick a "liar" base; the key is that the set of witnesses is large but not necessarily all bases.
- 5 (Answer T).** True: randomization affects expected behaviour; the worst-case pivot sequence still exists and yields $\Theta(n^2)$.
- 6 (Answer F).** False: the greedy-by-ratio method can be arbitrarily far from optimal for 0-1 instances; no universal constant-factor guarantee comes from that rule alone.
- 7 (Answer T).** True: DP works when optimal solutions contain optimal subsolutions (optimal substructure) and subproblems repeat (overlap); memoization caches them.
- 8 (Answer F).** False: pruning is a branch-and-bound/backtracking idea; DP typically evaluates a structured set of subproblems rather than discarding "branches" via bounds.
- 9 (Answer F).** False: Kruskal must reject any edge that creates a cycle, regardless of weight.
- 10 (Answer T).** True: a negative edge can produce a shorter path after a node was "finalized", breaking Dijkstra's correctness proof.
- 11 (Answer T).** True: Bloom filters may say "present" for absent items (FP), but if an item was inserted, all its bits are set so it will not be reported absent (no FN).
- 12 (Answer T).** True: more inserts set more bits to 1, increasing the chance that all k queried bits are 1 for an absent key.
- 13 (Answer T).** True: minimizing $(1 - e^{-kn/m})^k$ gives $k \approx (m/n) \ln 2$.
- 14 (Answer T).** True: counters allow decrementing on deletion; this increases memory and introduces counter overflow considerations.
- 15 (Answer F).** False: Cuckoo filters store short fingerprints (not full keys), so different keys can share a fingerprint, causing false positives.
- 16 (Answer F).** False: Count-Min overestimates because collisions only add extra counts; taking a minimum across rows keeps an upper bound, not a lower bound.
- 17 (Answer T).** True: each row gives an upper bound; the minimum across d rows reduces the effect of unlucky collisions.
- 18 (Answer T).** True: memory is $O(k)$ (or $O(1/\phi)$) depending on parameterization; it keeps a small set of counters for candidate heavy hitters.
- 19 (Answer T).** True: items with frequency $> n/k$ are guaranteed to appear among candidates; reported counts may still need verification/second pass for exact values.
- 20 (Answer F).** False: Morris counting is approximate; it stores a logarithmic-sized state X and uses probabilistic increments, not the exact count.